

Vector

Generated by Doxygen 1.13.2

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Vector< T, Allocator > Class Template Reference	5
4 File Documentation	7
4.1 Vector.h	7
Index	13

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Vector< T, Allocator >	5
--	---

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

Vector.h	7
------------------------------------	---

Chapter 3

Class Documentation

3.1 `Vector< T, Allocator >` Class Template Reference

Public Member Functions

- **Vector** (std::initializer_list< T > init)
- **Vector** (const [Vector](#) &other)
- **Vector** ([Vector](#) &&other) noexcept
- int **size** () const
- int **capacity** () const
- void **reserve** (size_t new_capacity)
- void **push_back** (const T &value)
- void **pop_back** ()
- T * **end** ()
- const T * **end** () const
- T * **begin** ()
- const T * **begin** () const
- T & **front** ()
- const T & **front** () const
- T & **back** ()
- const T & **back** () const
- void **resize** (int count, const T &value=T())
- void **insert** (int index, const T &value)
- bool **empty** () const
- void **shrink_to_fit** ()
- void **clear** ()
- void **erase** (int index)
- T & **at** (int index)
- const T & **at** (int index) const
- [Vector](#) & **operator=** (const [Vector](#) &other)
- [Vector](#) & **operator=** ([Vector](#) &&other) noexcept
- T & **operator[]** (int index)
- const T & **operator[]** (int index) const
- bool **operator==** (const [Vector](#) &other) const
- bool **operator!=** (const [Vector](#) &other) const
- bool **operator>** (const [Vector](#) &other) const
- bool **operator<** (const [Vector](#) &other) const
- bool **operator>=** (const [Vector](#) &other) const

- bool **operator<=** (const [Vector](#) &other) const
- void **swap** ([Vector](#) &other)
- void **assign** (const size_t count, const T &value)
- template<typename... Args>
void **emplace** (int index, Args &&...args)
- template<typename... Args>
void **emplace_back** (Args &&...args)

The documentation for this class was generated from the following file:

- [Vector.h](#)

Chapter 4

File Documentation

4.1 Vector.h

```
00001 #include <initializer_list>
00002 #include <memory>
00003 #include <stdexcept>
00004
00005 using std::out_of_range;
00006
00007 template <typename T, typename Allocator = std::allocator<T>
00008 class Vector
00009 {
00010 private:
00011     T *arr_;
00012     int capacity_;
00013     int size_;
00014     Allocator alloc_;
00015
00016 public:
00017     Vector()
00018     {
00019         capacity_ = 1;
00020         size_ = 0;
00021         arr_ = alloc_.allocate(capacity_);
00022     }
00023
00024     Vector(std::initializer_list<T> init)
00025     {
00026         size_ = static_cast<int>(init.size());
00027         capacity_ = size_ > 0 ? size_ : 1;
00028         arr_ = alloc_.allocate(capacity_);
00029
00030         int i = 0;
00031         for (const auto &val : init)
00032         {
00033             alloc_.construct(arr_ + i, val);
00034             i++;
00035         }
00036     }
00037
00038     Vector(const Vector &other)
00039     {
00040         capacity_ = other.capacity_;
00041         size_ = other.size_;
00042         arr_ = alloc_.allocate(capacity_);
00043         for (int i = 0; i < size_; i++)
00044         {
00045             alloc_.construct(arr_ + i, other.arr_[i]);
00046         }
00047     }
00048
00049     Vector(Vector &&other) noexcept : arr_(other.arr_), capacity_(other.capacity_), size_(other.size_)
00050     {
00051         other.arr_ = nullptr;
00052         other.capacity_ = 0;
00053         other.size_ = 0;
00054     }
00055
00056     ~Vector()
00057     {
```

```

00058         clear();
00059         alloc_.deallocate(arr_, capacity_);
00060     }
00061
00062     int size() const
00063     {
00064         return size_;
00065     }
00066
00067     int capacity() const
00068     {
00069         return capacity_;
00070     }
00071
00072     void reserve(size_t new_capacity)
00073     {
00074         if (new_capacity <= capacity_)
00075             return;
00076
00077         T *new_arr = alloc_.allocate(new_capacity);
00078         for (int i = 0; i < size_; i++)
00079         {
00080             alloc_.construct(new_arr + i, std::move(arr_[i]));
00081             alloc_.destroy(arr_ + i);
00082         }
00083         alloc_.deallocate(arr_, capacity_);
00084         arr_ = new_arr;
00085         capacity_ = new_capacity;
00086     }
00087
00088     void push_back(const T &value)
00089     {
00090         if (size_ == capacity_)
00091         {
00092             reserve(capacity_ * 2);
00093         }
00094         alloc_.construct(arr_ + size_, value);
00095         size_++;
00096     }
00097
00098     void pop_back()
00099     {
00100         if (size_ > 0)
00101         {
00102             alloc_.destroy(arr_ + size_ - 1);
00103             size_--;
00104         }
00105     }
00106
00107     T *end()
00108     {
00109         return arr_ + size_;
00110     }
00111
00112     const T *end() const
00113     {
00114         return arr_ + size_;
00115     }
00116
00117     T *begin()
00118     {
00119         return arr_;
00120     }
00121
00122     const T *begin() const
00123     {
00124         return arr_;
00125     }
00126
00127     T &front()
00128     {
00129         if (size_ > 0)
00130         {
00131             return arr_[0];
00132         }
00133         throw out_of_range("Vector is empty");
00134     }
00135
00136     const T &front() const
00137     {
00138         if (size_ > 0)
00139         {
00140             return arr_[0];
00141         }
00142         throw out_of_range("Vector is empty");
00143     }
00144

```

```
00145     T &back()
00146     {
00147         if (size_ > 0)
00148         {
00149             return arr_[size_ - 1];
00150         }
00151         throw out_of_range("Vector is empty");
00152     }
00153
00154     const T &back() const
00155     {
00156         if (size_ > 0)
00157         {
00158             return arr_[size_ - 1];
00159         }
00160         throw out_of_range("Vector is empty");
00161     }
00162
00163     void resize(int count, const T &value = T())
00164     {
00165         if (count < 0)
00166         {
00167             throw out_of_range("Size of vector can not be negative");
00168         }
00169         if (count == size_)
00170         {
00171             return;
00172         }
00173         else if (count < size_)
00174         {
00175             for (int i = count; i < size_; i++)
00176             {
00177                 alloc_.destroy(arr_ + i);
00178             }
00179             size_ = count;
00180         }
00181         else
00182         {
00183             if (count > capacity_)
00184             {
00185                 reserve(count);
00186             }
00187             for (int i = size_; i < count; i++)
00188             {
00189                 alloc_.construct(arr_ + i, value);
00190             }
00191             size_ = count;
00192         }
00193     }
00194
00195     void insert(int index, const T &value)
00196     {
00197         if (index < 0 || index > size_)
00198         {
00199             throw out_of_range("Index out of range");
00200         }
00201
00202         if (size_ == capacity_)
00203         {
00204             reserve(capacity_ * 2);
00205         }
00206
00207         for (int i = size_; i > index; i--)
00208         {
00209             alloc_.construct(arr_ + i, std::move(arr_[i - 1]));
00210             alloc_.destroy(arr_ + i - 1);
00211         }
00212         alloc_.construct(arr_ + index, value);
00213         size_++;
00214     }
00215
00216     bool empty() const
00217     {
00218         return size_ == 0;
00219     }
00220
00221     void shrink_to_fit()
00222     {
00223         if (capacity_ == size_)
00224             return;
00225
00226         T *new_arr = alloc_.allocate(size_);
00227         for (int i = 0; i < size_; i++)
00228         {
00229             alloc_.construct(new_arr + i, std::move(arr_[i]));
00230             alloc_.destroy(arr_ + i);
00231         }
00232     }
```

```

00232         alloc_.deallocate(arr_, capacity_);
00233         arr_ = new_arr;
00234         capacity_ = size_;
00235     }
00236
00237     void clear()
00238     {
00239         for (int i = 0; i < size_; i++)
00240         {
00241             alloc_.destroy(arr_ + i);
00242         }
00243         size_ = 0;
00244     }
00245
00246     void erase(int index)
00247     {
00248         if (index < 0 || index >= size_)
00249         {
00250             throw out_of_range("Index out of range");
00251         }
00252         alloc_.destroy(arr_ + index);
00253         for (int i = index; i < size_ - 1; i++)
00254         {
00255             alloc_.construct(arr_ + i, std::move(arr_[i + 1]));
00256             alloc_.destroy(arr_ + i + 1);
00257         }
00258         size_--;
00259     }
00260
00261     T &at(int index)
00262     {
00263         if (index < 0 || index >= size_)
00264         {
00265             throw out_of_range("Index out of range");
00266         }
00267         return arr_[index];
00268     }
00269
00270     const T &at(int index) const
00271     {
00272         if (index < 0 || index >= size_)
00273         {
00274             throw out_of_range("Index out of range");
00275         }
00276         return arr_[index];
00277     }
00278
00279     Vector &operator=(const Vector &other)
00280     {
00281         if (this != &other)
00282         {
00283             clear();
00284             alloc_.deallocate(arr_, capacity_);
00285             capacity_ = other.capacity_;
00286             size_ = other.size_;
00287             arr_ = alloc_.allocate(capacity_);
00288             for (int i = 0; i < size_; i++)
00289             {
00290                 alloc_.construct(arr_ + i, other.arr_[i]);
00291             }
00292         }
00293         return *this;
00294     }
00295
00296     Vector &operator=(Vector &&other) noexcept
00297     {
00298         if (this != &other)
00299         {
00300             clear();
00301             alloc_.deallocate(arr_, capacity_);
00302             arr_ = other.arr_;
00303             capacity_ = other.capacity_;
00304             size_ = other.size_;
00305             other.arr_ = nullptr;
00306             other.capacity_ = 0;
00307             other.size_ = 0;
00308         }
00309         return *this;
00310     }
00311
00312     T &operator[](int index)
00313     {
00314         if (index < 0 || index >= size_)
00315         {
00316             throw out_of_range("Index out of range");
00317         }
00318         return arr_[index];

```

```

00319     }
00320
00321     const T &operator[](int index) const
00322     {
00323         if (index < 0 || index >= size_)
00324         {
00325             throw out_of_range("Index out of range");
00326         }
00327         return arr_[index];
00328     }
00329
00330     bool operator==(const Vector &other) const
00331     {
00332         if (size_ != other.size_)
00333         {
00334             return false;
00335         }
00336
00337         for (int i = 0; i < size_; i++)
00338         {
00339             if (arr_[i] != other[i])
00340             {
00341                 return false;
00342             }
00343         }
00344
00345         return true;
00346     }
00347
00348     bool operator!=(const Vector &other) const
00349     {
00350         return !(*this == other);
00351     }
00352
00353     bool operator>(const Vector &other) const
00354     {
00355         int min_size = (size_ < other.size_) ? size_ : other.size_;
00356
00357         for (int i = 0; i < min_size; i++)
00358         {
00359             if (arr_[i] > other[i])
00360             {
00361                 return true;
00362             }
00363             else if (arr_[i] < other[i])
00364             {
00365                 return false;
00366             }
00367         }
00368
00369         return size_ > other.size_;
00370     }
00371
00372     bool operator<(const Vector &other) const
00373     {
00374         int min_size = (size_ < other.size_) ? size_ : other.size_;
00375
00376         for (int i = 0; i < min_size; ++i)
00377         {
00378             if (arr_[i] < other[i])
00379             {
00380                 return true;
00381             }
00382             if (arr_[i] > other[i])
00383             {
00384                 return false;
00385             }
00386         }
00387
00388         return size_ < other.size_;
00389     }
00390
00391     bool operator>=(const Vector &other) const
00392     {
00393         return !(*this < other);
00394     }
00395
00396     bool operator<=(const Vector &other) const
00397     {
00398         return !(*this > other);
00399     }
00400
00401     void swap(Vector &other)
00402     {
00403         std::swap(arr_, other.arr_);
00404         std::swap(capacity_, other.capacity_);
00405         std::swap(size_, other.size_);

```

```
00406     }
00407
00408 void assign(const size_t count, const T &value)
00409 {
00410     if (count > capacity_)
00411     {
00412         reserve(count);
00413     }
00414
00415     for (int i = 0; i < count; i++)
00416     {
00417         if (i < size_)
00418         {
00419             arr_[i] = value;
00420         }
00421         else
00422         {
00423             alloc_.construct(arr_ + i, value);
00424         }
00425     }
00426     size_ = count;
00427 }
00428
00429 template <typename... Args>
00430 void emplace(int index, Args &&...args)
00431 {
00432     if (index < 0 || index > size_)
00433     {
00434         throw std::out_of_range("Index out of range");
00435     }
00436
00437     if (size_ == capacity_)
00438     {
00439         reserve(capacity_ * 2);
00440     }
00441
00442     for (int i = size_; i > index; i--)
00443     {
00444         alloc_.construct(arr_ + i, std::move(arr_[i - 1]));
00445         alloc_.destroy(arr_ + i - 1);
00446     }
00447
00448     alloc_.construct(arr_ + index, T(std::forward<Args>(args)...));
00449     size_++;
00450 }
00451
00452 template <typename... Args>
00453 void emplace_back(Args &&...args)
00454 {
00455     if (size_ == capacity_)
00456     {
00457         reserve(capacity_ * 2);
00458     }
00459
00460     alloc_.construct(arr_ + size_, T(std::forward<Args>(args)...));
00461     size_++;
00462 }
00463 };
```


Index

Vector< T, Allocator >, [5](#)